

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO5 – Articulation	Assessment:	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	YUVARAJ R	Reg. No.:	192524058
Section / Batch:	2026	Date:	

Code of Conduct

I YUVARAJ R / 192524058 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Yuvaraj R

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Reference Resolution Using Multiple Constraints

Design an algorithm to perform **Reference Resolution** by identifying pronouns and selecting the most appropriate antecedent using gender, number, recency, and semantic compatibility.

Apply your algorithm to the discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Generate possible antecedents for each pronoun.
3. Apply gender, number, recency, and semantic constraints.
4. Select the most appropriate antecedent.
5. Generate the resolved discourse.

Finally, explain how your algorithm avoids incorrect resolution when multiple antecedents have similar grammatical properties.

2. Algorithm for Entity Chains and Discourse Coherence

Design an algorithm to identify **entity chains** in a discourse and use them to evaluate text coherence.

Apply your algorithm to the following text:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Link expressions referring to the same entity.
3. Construct entity chains.
4. Calculate or estimate the continuity of entities across sentences.
5. Determine whether the discourse is coherent.

Finally, explain how breaking an entity chain could reduce discourse coherence.

3. Algorithm for Identifying Discourse Relations

Design an algorithm to identify **discourse relations** between consecutive sentences using discourse markers and contextual meaning.

Apply your algorithm to:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Your algorithm should identify relations such as:

- Cause–Effect
- Sequence
- Problem–Solution
- Elaboration

Perform the following steps:

1. Divide the discourse into individual discourse units.
2. Identify explicit discourse markers.
3. Analyze the semantic relationship between units.
4. Assign an appropriate discourse relation.
5. Construct a discourse structure representation.

Finally, justify how the identified relations make the discourse logically coherent.

4. Algorithm for Dialogue Act Classification

Design an algorithm to recognize **Dialogue Acts** in a conversation.

Apply your algorithm to:

User: "Hello, I need help with my assignment."

Agent: "Sure, what problem are you facing?"

User: "I do not understand machine learning."

Agent: "I can explain the basic concepts to you."

Classify each utterance into one of the following:

- Greeting
- Request
- Question
- Inform
- Offer
- Confirmation

Perform the following steps:

1. Preprocess each utterance.
2. Extract relevant linguistic features.
3. Identify the communicative intention.
4. Assign a dialogue act.
5. Generate the dialogue-act sequence.

Finally, explain how dialogue-act recognition helps a conversational agent select an appropriate next response.

5. Algorithm for Dialogue State Tracking

Design an algorithm for a **Conversational Agent** that maintains dialogue state while collecting information from a user.

Apply your algorithm to:

User: "I want to book a flight."

Agent: "Where would you like to travel?"

User: "I want to go to Delhi."

Agent: "From which city?"

User: "From Chennai."

Your algorithm should maintain the following slots:

Departure City

Destination City

Travel Date

Number of Passengers

Perform the following steps:

1. Initialize the dialogue state.
2. Identify information provided in each utterance.
3. Update the corresponding slot.
4. Identify missing information.
5. Generate the next appropriate question.

Finally, explain how dialogue state tracking improves coherence in a multi-turn conversation.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach

Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1	3	3	3	3	4	16	80
2	3	3	3	3	4	16	
3	3	3	3	3	4	16	
4	3	3	3	3	4	16	
5	3	3	3	3	4	16	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Reference Resolution Using Multiple Constraints

1. Entities and referring expressions

Type	Expressions
Entities	Meena, Kavya, laptop, project
Referring expressions	she, one, She, her, it

2. Possible antecedents and constraint analysis

Expression	Possible antecedents	Selected antecedent	Reason
she	Meena / Kavya	Kavya	Both match gender/number; semantic relation “needed one for her project” explains why Meena gave Kavya the laptop.
one	laptop	laptop	“one” refers to an object of the laptop type.
She	Meena / Kavya	Kavya	Recency and discourse continuity favor Kavya.
her	Meena / Kavya	Meena	In “She thanked her”, subject Kavya thanks the giver, Meena.
it	laptop / project	laptop	Only the laptop is semantically compatible with “started using it”.

3. Pseudocode

ALGORITHM ResolveReferences(discourse)

1. Extract entities and pronouns from discourse.
2. FOR each pronoun p:
 - a. Generate antecedent candidates occurring before p.
 - b. Remove candidates that fail number agreement.
 - c. Remove candidates that fail gender agreement.
 - d. Score remaining candidates using recency.
 - e. Check semantic compatibility with the predicate and context.
 - f. Check grammatical role and discourse coherence.
 - g. Select the candidate with the highest valid score.
3. Replace each pronoun with its selected antecedent.
4. RETURN resolved discourse.

4. Resolved discourse

Meena gave Kavya a laptop because Kavya needed a laptop for Kavya’s project. Kavya thanked Meena and started using the laptop.

5. Avoiding incorrect resolution

When multiple antecedents have similar grammatical properties, the algorithm does not depend only on gender or recency. It combines grammatical agreement with semantic compatibility, predicate roles and discourse coherence. Therefore, “her” is resolved to Meena because Kavya naturally thanks the giver, while “it” is resolved to the laptop because a laptop can be used.

Q2. Algorithm for Entity Chains and Discourse Coherence

1. Entities and referring expressions

Expression	Entity
Anjali / She / Anjali	Person: Anjali
a new laptop / The laptop / it / the computer	Device: laptop
a powerful processor	Processor
Python	Software
a machine learning application	Application

2. Entity chains

Chain 1: Anjali → She → Anjali

Chain 2: a new laptop → The laptop → it → the computer

3. Pseudocode

ALGORITHM EvaluateCoherence(text)

1. Split text into sentences.
2. Extract named entities, noun phrases and referring expressions.
3. FOR each expression e:
 - a. Compare e with existing entity chains.
 - b. Check lexical match, synonymy, pronoun agreement and semantic compatibility.
 - c. IF a compatible chain exists, link e to that chain.
 - d. ELSE create a new entity chain.
4. FOR each adjacent sentence pair, count shared/continued entities.
5. Estimate continuity = sentence transitions with continued entities / total sentence transitions.
6. IF continuity is high and chains are logically consistent, label discourse COHERENT.
7. RETURN entity chains, continuity and coherence label.

4. Continuity and coherence

The laptop entity appears in every sentence through “laptop”, “The laptop”, “it”, and “the computer”. Anjali also reappears through “She” and “Anjali”. Thus, important entities are continuously maintained across the discourse. The estimated continuity is high, so the discourse is coherent.

5. Effect of breaking an entity chain

If “it” or “the computer” were incorrectly linked to another entity, the reader could lose track of the laptop. This would weaken continuity between sentences and make the text less coherent.

Q3. Algorithm for Identifying Discourse Relations

1. Discourse units

U1: The server crashed unexpectedly.

U2: As a result, users could not access the application.

U3: The technical team restarted the server.

U4: After that, the system became available again.

2. Markers and relations

Units	Marker / Context	Relation
U1 → U2	As a result	Cause–Effect
U2 → U3	Context: outage followed by corrective action	Problem–Solution
U3 → U4	After that	Sequence; also Cause–Effect

3. ALGORITHM IdentifyDiscourseRelations(text)

1. Segment text into discourse units U1...Un.
2. FOR each consecutive pair (Ui, Ui+1):
 - a. Detect explicit markers such as “as a result”, “because”, “after that”.
 - b. Map known markers to candidate discourse relations.
 - c. Analyze events and semantic context when no explicit marker exists.
 - d. Assign Cause–Effect, Sequence, Problem–Solution or Elaboration.
3. Create nodes for discourse units and labeled edges for relations.
4. RETURN discourse structure.

4. Discourse structure representation

Server crash (U1) --Cause–Effect→ Application unavailable (U2) --Problem–Solution→ Server restarted (U3) --Sequence/Cause–Effect→ System available (U4)

5. Justification

The relations create a logical progression: a crash causes loss of access, the technical team responds to the problem by restarting the server, and after the restart the system becomes available. The explicit markers reinforce this causal and temporal structure.

Q4. Algorithm for Dialogue Act Classification

1. Dialogue-act classification

Utterance	Dialogue Act	Reason
User: “Hello, I need help with my assignment.”	Greeting + Request	“Hello” greets; “I need help” requests assistance.
Agent: “Sure, what problem are you facing?”	Confirmation + Question	“Sure” acknowledges willingness; the sentence asks for information.
User: “I do not understand machine learning.”	Inform	The user states a problem/state of knowledge.
Agent: “I can explain the basic concepts to you.”	Offer	The agent offers assistance.

2. Pseudocode

ALGORITHM ClassifyDialogueActs(conversation)

1. FOR each utterance u:
 - a. Normalize and tokenize u.
 - b. Extract keywords, punctuation, sentence form and speaker information.
 - c. Detect greeting words such as hello/hi.
 - d. Detect request patterns such as need/help/please.
 - e. Detect question forms and question marks.
 - f. Detect statements that provide information.
 - g. Detect offer patterns such as “I can...” or “Would you like...”.
 - h. Detect confirmation/acknowledgement words such as sure/yes/okay.
 - i. Assign the most appropriate dialogue act(s).
2. RETURN dialogue-act sequence.

3. Dialogue-act sequence

Greeting + Request → Confirmation + Question → Inform → Offer

4. Importance

Dialogue-act recognition tells the conversational agent what communicative function each utterance performs. It can therefore choose an appropriate next response, such as answering a question, fulfilling a request, asking for missing information, or responding to an inform statement.

Q5. Algorithm for Dialogue State Tracking

1. Initial dialogue state

Slot	Initial value
Departure City	Unknown
Destination City	Unknown
Travel Date	Unknown
Number of Passengers	Unknown

2. Pseudocode

ALGORITHM TrackDialogueState(conversation)

1. Initialize all required slots as UNKNOWN.
2. FOR each user utterance u:
 - a. Detect user intent.
 - b. Extract slot-value pairs from u.
 - c. Update corresponding slots in the dialogue state.
 - d. Identify slots that are still UNKNOWN.
 - e. IF required information is missing, ask for the next missing slot.
3. When all required slots are filled, confirm the complete booking information.
4. RETURN final dialogue state.

3. State updates for the given conversation

Turn	Information extracted	Dialogue state after update
“I want to book a flight.”	Intent = Book Flight	All four slots unknown
“I want to go to Delhi.”	Destination City = Delhi	Destination = Delhi
“From Chennai.”	Departure City = Chennai	Departure = Chennai; Destination = Delhi

4. Missing information and next question

After the given conversation, Travel Date and Number of Passengers are still missing. The next appropriate question is: “What date would you like to travel?” After receiving the date, the agent should ask: “How many passengers will be travelling?”

5. Final state at the current point

Slot	Current value
Departure City	Chennai
Destination City	Delhi
Travel Date	Unknown
Number of Passengers	Unknown

6. Importance of dialogue state tracking

Dialogue state tracking preserves information collected across multiple turns. The agent therefore does not repeatedly ask for Delhi or Chennai and can focus only on missing information. This improves coherence, efficiency and continuity in a multi-turn conversation.